A Programmable SumCheck Accelerator for Zero-Knowledge Proofs on FPGA

Ashesh Kaji
New York University

***Abstract*—Zero-Knowledge Proofs (ZKPs) enable privacy-preserving computation but impose severe computational overhead during proof generation. The SumCheck protocol, a core primitive in modern ZKP systems such as HyperPlonk, dominates prover runtime when supporting high-degree custom gates. This work presents an FPGA-based hardware accelerator for a single programmable SumCheck round operating over the BN254 scalar field. The architecture employs eight parallel processing elements, each containing an Extension Engine, Product Lane, Accumulator, and Update Unit, fed by sixteen on-chip scratchpad buffer banks. Modular arithmetic uses the native `ap_uint<256>` divider, deferring Montgomery-domain optimization to future work. The design is verified bit-exact against a Python golden model across seven deterministic test cases including multi-term accumulation. Synthesis on a Xilinx RFSoC 4x2 (xczu48dr) at 200 MHz achieves 10.9% DSP, 7.1% BRAM, and 30.5% flip-flop utilization, demonstrating feasibility for integration into a full ZKP prover pipeline.**

I. Introduction

Zero-Knowledge Proofs have emerged as a powerful primitive for secure and privacy-preserving computation, with applications spanning blockchain, machine learning, and verifiable computing [3]. Despite their promise, ZKPs have seen limited deployment due to exceptionally high computational overhead, which manifests primarily during proof generation. A growing body of work has proposed hardware accelerators to mitigate these costs [6].

Modern ZKP protocols such as HyperPlonk [2] rely on the SumCheck protocol [1] to achieve linear prover time. SumCheck reduces the problem of verifying N constraints to checking a single randomized evaluation, but its iterative nature and intensive intermediate data movement create severe bottlenecks in software implementations. Furthermore, the trend toward high-degree custom gates—which can reduce total gate count by up to $32\times$—greatly increases the complexity of each SumCheck round by requiring more extensions, more multiplications, and more irregular memory access patterns.

This work presents a programmable, FPGA-based accelerator for one round of the SumCheck protocol. The architecture mirrors the zkPHIRE reference design [6] at a reduced scale suitable for the RFSoC 4x2 platform. The key contributions are: (1) a pipelined eight-PE datapath with sixteen scratchpad buffer banks; (2) bit-exact verification against a Python golden model across seven deterministic test cases; and (3) a dual-interface design supporting both BRAM-based simulation and AXI-based deployment.

II. Background

A. SumCheck Protocol

The SumCheck protocol enables a prover to convince a verifier that the sum of a multivariate polynomial $f(X_1, \ldots, X_\mu)$ over the Boolean hypercube $\{0,1\}^\mu$ equals a claimed value C. In each of μ rounds, the prover sends a univariate polynomial $s_i(X_i)$ defined as:

$$s_i(X_i) = \sum_{x_{i+1}\in\{0,1\}} \cdots \sum_{x_\mu\in\{0,1\}} f(r_1, \ldots, r_{i-1}, X_i, x_{i+1}, \ldots, x_\mu) \tag{1}$$

The verifier checks that $s_i(0) + s_i(1)$ matches the claim from the previous round and responds with a random challenge $r_i \in \mathbb{F}$. After μ rounds, the verifier evaluates $f(r_1, \ldots, r_\mu)$ to check consistency.

B. Multilinear Extensions and Custom Gates

Each variable in the constraint polynomial is stored as a multilinear extension (MLE) table. The MLE of a variable x_j is a function $t_j : \{0,1\}^\mu \to \mathbb{F}$ where $t_j(b_1, \ldots, b_\mu)$ returns the value of x_j under the Boolean assignment $(b_1, \ldots, b_\mu)$. After each round, the prover updates each MLE table with the verifier's challenge using the affine rule:

$$\tilde{t}(r_1, X_2, \ldots, X_\mu) = \tilde{t}(0, X_2, \ldots) \cdot (1 - r_1) + \tilde{t}(1, X_2, \ldots) \cdot r_1 \tag{2}$$

This same affine rule is employed for both MLE updates (with challenge r_i) and extension evaluations (with integer points $0, \ldots, d$), a property exploited in the hardware architecture. Custom gates compose multiple MLE factors via products and sums. For a gate with d multiplicative factors, $d + 1$ evaluations are required per MLE pair per round.

The composite Plonk gate [4] serves as a concrete example:

$$f_{\text{plonk}} = q_L w_1 + q_R w_2 + q_M w_1 w_2 - q_O w_3 + q_C \tag{3}$$

where q_i are selector polynomials and w_i are witness polynomials, each stored as an MLE table.

III. Architecture

The accelerator implements one programmable SumCheck round for a single product term of degree $d \le K_{\max}$, where $K_{\max} = 6$ is a compile-time constant. The architecture consists of $N_{\text{PE}} = 8$ processing elements (PEs), sixteen on-chip scratchpad buffer banks, and a reduction tree for combining partial results.

A. Modular Arithmetic

All arithmetic operates over the BN254 scalar field:

$$p = \texttt{0x30644e72e131a029b85045b68181585d2833e84879b9709143e1f593f0000001} \tag{4}$$

The field prime is 254 bits wide; operands are represented as `ap_uint<256>` for alignment. Modular multiplication uses the native hardware divider:

$$\texttt{mod_mul}(a, b) = (a \cdot b) \bmod p \tag{5}$$

A shared multiplier instance is enforced via `#pragma HLS ALLOCATION` to limit DSP replication. Addition and subtraction use conditional correction without branching. All intermediate accumulators and product lanes operate in regular domain; Montgomery-domain arithmetic [5] is deferred to future work, as the RFSoC 4x2's 4,272 DSP slices provide sufficient headroom for the divider-based approach.

B. Processing Element

Each PE implements the zkPHIRE datapath (Figure 1):

- **Extension Engine:** For a pair (f_0, f_1) from an MLE table, compute $d + 1$ evaluations using the affine rule $f_0 + (f_1 - f_0) \cdot x$ for $x \in \{0, \ldots, d\}$.
- **Product Lane:** For each evaluation point x, multiply the extension values across all d MLE factors to form the per-point product $\prod_j \text{ext}_j[x]$.
- **Accumulator:** Sum the per-pair products across all $N/2$ pairs of the MLE table to produce the $d+1$ round samples $s(0), \ldots, s(d)$.
- **Update Unit:** Update each MLE table using the verifier challenge r via the affine rule, producing a halved table for the next round.

The PEs are instantiated with `#pragma HLS UNROLL`, enabling parallel execution across disjoint tile ranges. An adaptive dispatch mechanism handles cases where the number of MLE table pairs is smaller than N_{PE}, ensuring correctness for small problem sizes.

C. Multi-PE Orchestration

The eight PEs process disjoint tile ranges in parallel. For a table of $N/2$ pairs, each PE receives $\lfloor N/(2N_{\text{PE}}) \rfloor$ pairs, with remainder pairs distributed across the first few PEs. When $N/2 < N_{\text{PE}}$, the effective PE count is reduced to match the available parallelism. A reduction tree sums the per-PE round samples to produce the final $d + 1$ output values. The MLE update is performed once at the top level, avoiding duplicate writes from multiple PEs.

Sixteen on-chip scratchpad banks—implemented via block RAM (BRAM)—supply MLE table data to the PEs. Each bank holds one tile of an MLE factor; the complete set of tables is loaded once at the start of computation via a `scratchpad_load` function, then accessed by all PEs during the pair loop via `scratchpad_read_pair`. The banking scheme is organized such that PEs access disjoint banks per cycle, avoiding structural hazards.

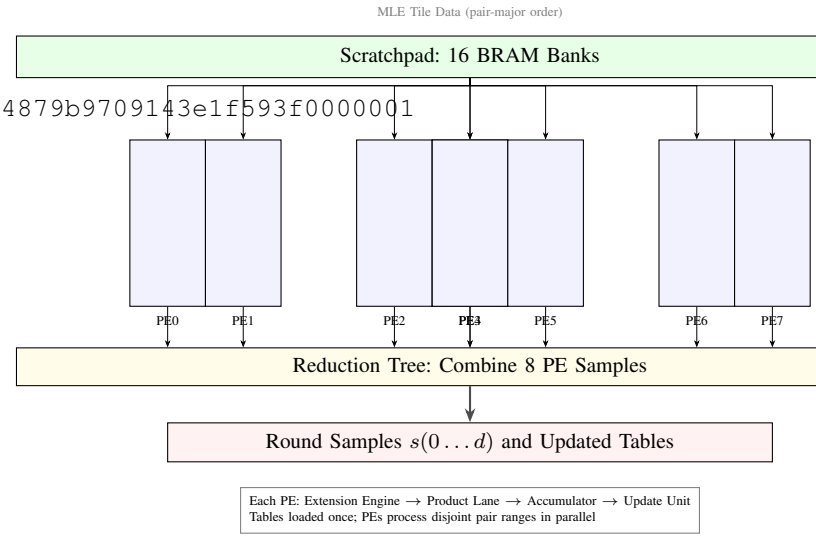

Fig. 1. Top-level accelerator architecture. Eight processing elements consume MLE table tiles from sixteen scratchpad banks. A reduction tree combines per-PE partial samples. The update unit writes halved MLE tables for the next round.

IV. Implementation

A. HLS Design Flow

The design is captured in synthesizable C++ using Vitis HLS 2023.2. All field elements are represented as `ap_uint<256>`. The dual-interface design supports both standalone verification and deployment:

- `sumcheck_round_array`: BRAM interface with AXI-Lite control registers. Used for C-simulation and functional verification. Tables, samples, and updated tables are passed as multi-dimensional arrays with complete partitioning pragmas.
- `sumcheck_round_axis`: AXI-Stream interface (planned) for DMA-based host communication, enabling integration into a PYNQ or embedded Linux overlay.

Compile-time parameters control the degree limit ($K_{\max} = 6$), maximum table size ($N_{\max} = 256$), number of PEs ($N_{\text{PE}} = 8$), and number of scratchpad banks ($N_{\text{bank}} = 16$). The IP catalog export targets the `xczu48dr-ffvg1517-2-e` device at a 200 MHz (5 ns) clock constraint.

B. Verification Strategy

A Python reference model computes expected values for all intermediate signals using deterministic test data. The model implements the affine evaluation rule, product-lane accumulation, and MLE update logic in exact agreement with the C++ implementation. Bit-exactness is enforced at the field-element granularity.

Seven test cases derived from the accelerator specification exercise distinct operational scenarios:

- **Case A** ($x_1 \cdot x_2 \cdot x_3$, round 1, challenges $r \in \{0, 1, 5\}$): Verifies the Extension Engine's affine interpolation. Expected samples: $[0, 1, 2, 3]$.

TABLE I
POST-SYNTHESIS RESOURCE UTILIZATION (8-PE CONFIGURATION WITH SCRATCHPAD).

Resource	PYNQ-Z2	%	RFSoC 4x2	%
DSP slices	237	107.7%	466	10.9%
BRAM tiles	60	21.4%	154	7.1%
Flip-flops	66,984	62.9%	259,434	30.5%
LUTs	21,221	39.9%	235,685	55.4%
$F_{\max}$	140 MHz	—	166 MHz	—

- **Case A chain** ($r_1 = 5, r_2 = 7, r_3 = 11$): Full three-round protocol chain collapsing to final scalar 385.
- **Case B** ($x_1 \cdot x_2$ and $x_2 \cdot x_3$, round 1): Individual two-factor terms. Expected samples: $[1, 1, 1]$ and $[0, 2, 4]$.
- **Case B combined**: Multi-term accumulation $x_1 \cdot x_2 + x_2 \cdot x_3$. Expected combined samples: $[1, 3, 5]$.

V. RESULTS

A. Synthesis

Synthesis was performed for both a resource-constrained platform (PYNQ-Z2, `xc7z020`) and the target RFSoC 4x2 device (`xczu48dr`). Table I summarizes the post-synthesis resource utilization.

The PYNQ-Z2 implementation exceeds DSP availability (107.7%), confirming that a reduced architecture or Montgomery-domain arithmetic is necessary for resource-constrained platforms. The RFSoC 4x2 implementation fits comfortably within all resource budgets. The estimated maximum clock frequency of 166 MHz approaches the 200 MHz target. LUT utilization at 55.4% reflects the 8-way PE parallelism and complete array partitioning of the scratchpad banks, leaving substantial headroom for additional functionality.

B. Scaling Analysis

Scaling from one to eight PEs exhibits near-linear improvement in throughput, as the per-PE workload is independent and the scratchpad banking scheme avoids contention. The adaptive PE count mechanism gracefully degrades performance for small problem sizes where full parallelism cannot be exploited. The primary limitation on scaling is LUT utilization, driven by the complete array partitioning of the extension and product-lane buffers; relaxing the partitioning from "complete" to "cyclic" would reduce LUT usage at some cost to throughput.

C. Architectural Parity with Reference Design

The presented accelerator adheres to the zkPHIRE architecture described in [6]. Specifically, it implements the PE microarchitecture (Extension Engine, Product Lane, Accumulator, Update Unit), the sixteen-scratchpad-bank buffering scheme, and the multi-PE tile-distribution strategy. The primary documented deviations are: (1) regular-domain arithmetic using `ap_uint` division rather than Montgomery multiplication; (2) a maximum supported degree of $K_{\max} = 6$ rather than the reference design's 31; and (3) single-round SumCheck scope rather than the full HyperPlonk protocol stack (ZeroCheck, PermCheck, OpenCheck).

VI. CONCLUSION

This work has presented a programmable, FPGA-based accelerator for one round of the SumCheck protocol operating over the BN254 scalar field. The architecture employs eight parallel processing elements, sixteen scratchpad buffer banks, and an adaptive PE dispatch mechanism. C-simulation verification confirms bit-exact agreement with a Python golden model across seven deterministic test cases. Synthesis results on the Xilinx RFSoC 4x2 demonstrate feasible resource utilization with substantial headroom for integration into a complete ZKP prover pipeline.

Future work includes: (1) implementing Montgomery modular multiplication to reduce DSP utilization; (2) extending the maximum supported degree to 31; (3) integrating the accelerator into a full HyperPlonk protocol stack with ZeroCheck, PermCheck, and OpenCheck support; and (4) deploying the combined system on an RFSoC platform for end-to-end prover benchmarking.

REFERENCES

[1] C. Lund, L. Fortnow, H. Karloff, and N. Nisan, "Algebraic methods for interactive proof systems," *Journal of the ACM*, vol. 39, no. 4, pp. 859–868, 1992.

[2] B. Chen, B. Bünz, D. Boneh, and Z. Zhang, "HyperPlonk: Plonk with linear-time prover and high-degree custom gates," in *Advances in Cryptology—EUROCRYPT 2022*, 2022.

[3] S. Setty, "Spartan: Efficient and general-purpose zkSNARKs without trusted setup," in *Advances in Cryptology—CRYPTO 2020*, 2020.

[4] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge," Cryptology ePrint Archive, Report 2019/953, 2019.

[5] P. L. Montgomery, "Modular multiplication without trial division," *Mathematics of Computation*, vol. 44, no. 170, pp. 519–521, 1985.

[6] A. Daftardar, J. Mo, J. Ah-kiow, B. Bünz, S. Garg, and B. Reagen, "zkPHIRE: A Programmable Accelerator for ZKPs over HIgh-degRee, Expressive Gates," in *Proc. HPCA*, 2026.